

Name: - Shubham Das

S. A. P I. D: - 590033500

Date: - 24/09/2025

Programme: - B. TECH (CSE)

Assignment: - Write a C program to perform the following menu-driven operations on a given 2D array:

1. Find the sum of all elements in a matrix.
2. Find the sum of each row of a matrix and store it in an array.
3. Find the transpose of a matrix.
4. Exit

Sample Output:

Enter the number of rows and columns: 2 3

Enter the matrix elements:

1 2 3

4 5 6

Enter your choice: 1

Sum of all elements = 21

Enter your choice: 2

Row sums:

Sum of row 1 = 6

Sum of row 2 = 15

The row-sum array contains: [6, 15]

Enter your choice: 3

Transpose of the matrix:

1 4

2 5

3 6

Part 1: Thorough understanding of the problem statement

Input: -

1. *rows, cols* (Matrix dimensions, Variable: *int*, Unit: unitless integer)

2. *matrix[rows][cols]* (2D array elements, Variable: *int*, Unit: scalar values)

3. *choice* (Menu option, Variable: *int*, Unit: unitless integer)

Output: -

1. Total sum of all elements (Variable: *int*, Unit: scalar value)

2. Sum of each row (Variables: *int* array elements, Unit: scalar values)

3. Transpose of the matrix (Variables: *int* 2D array, Unit: scalar values)

Formula

1. Total Sum: $total_sum += matrix[i][j];$
2. Row Sum: $row_sum += matrix[i][j];$
3. Transpose: $transpose[j][i] = matrix[i][j];$

Part 2: Writing, compiling and evaluating the algorithm

Step 1:- Start.

Step 2:- Declare integer variables *rows*, *cols*, *i*, *j*, *choice*, and *total_sum*.

Step 3:- Prompt the user to enter the number of rows and columns. Read values into *rows* and *cols*.

Step 4:- Declare C99 Variable Length Arrays (VLAs): *matrix[rows][cols]*, *row_sum[rows]*, and *transpose[cols][rows]*.

Step 5:- Prompt the user to input the matrix elements. Use nested loops to read values into *matrix[i][j]*.

Step 6:- Initiate a *do-while* loop to display the menu and accept *choice*.

Step 7:- Evaluate *choice* using a *switch* statement:

Case 1 (Sum of all elements): - Initialize *total_sum* = 0. Use nested loops to iterate through matrices. Add each element to *total_sum*.

Print *total_sum*.

Case 2 (Row Sums): - Loop *i* through *rows*. Initialize *row_sum[i]* = 0. Loop *j* through *cols*, adding *matrix[i][j]* to *row_sum[i]*. Print

the individual row sums and the final *row_sum* array.

Case 3 (Transpose): Loop *i* through *rows* and *j* through *cols*. Assign *transpose[j][i]* = *matrix[i][j]*. Print the *transpose* matrix

(looping outer *cols*, inner *rows*).

Case 4 (Exit): Terminate the loop.

Default: Print an invalid choice error.

Step 8:- Stop.

Part 3: - Writing and compiling the C program

GNU nano 9.0

```
#include <stdio.h>

int main() {
    int rows, cols, i, j, choice;

    // Read dimensions
    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Declare Variable Length Arrays based on input
    int matrix[rows][cols];
    int row_sum[rows];
    int transpose[cols][rows];

    // Read matrix elements
    printf("Enter the matrix elements:\n");
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Menu loop
    printf("\nMenu:\n1. Sum of all elements\n2. Sum of each row\n3. Transpose\n4. Exit\n");
    do {
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: {
                int total_sum = 0;
                for (i = 0; i < rows; i++) {
                    for (j = 0; j < cols; j++) {
                        total_sum += matrix[i][j];
                    }
                }
                printf("Sum of all elements = %d\n", total_sum);
                break;
            }
            case 2: {
                printf("Row sums:\n");
                for (i = 0; i < rows; i++) {
                    row_sum[i] = 0; // Initialize sum for current row
                    for (j = 0; j < cols; j++) {
                        row_sum[i] += matrix[i][j];
                    }
                    printf("Sum of row %d = %d\n", i + 1, row_sum[i]);
                }
            }
        }
    }
}
```

```
        case 2: {
            printf("Row sums:\n");
            for (i = 0; i < rows; i++) {
                row_sum[i] = 0; // Initialize sum for current row
                for (j = 0; j < cols; j++) {
                    row_sum[i] += matrix[i][j];
                }
                printf("Sum of row %d = %d\n", i + 1, row_sum[i]);
            }

            // Print the array format
            printf("The row-sum array contains: [");
            for (i = 0; i < rows; i++) {
                printf("%d", row_sum[i]);
                if (i < rows - 1) printf(", ");
            }
            printf("]\n");
            break;
        }
        case 3: {
            // Compute transpose
            for (i = 0; i < rows; i++) {
                for (j = 0; j < cols; j++) {
                    transpose[j][i] = matrix[i][j];
                }
            }

            // Print transpose
            printf("Transpose of the matrix:\n");
            for (i = 0; i < cols; i++) {
                for (j = 0; j < rows; j++) {
                    printf("%d\t", transpose[i][j]);
                }
                printf("\n");
            }
            break;
        }
        case 4:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice. Please try again.\n");
    }
} while (choice != 4);

return 0;
```

}

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

Step 1: - Open the terminal

Step 2: - Create the source code

Step 3: - Write and Save Code

Step 5: - Run the Binary Executable program

```
(kali㉿kali)-[~]
$ nano array_ops.c

(kali㉿kali)-[~]
$ gcc array_ops.c -o array_ops

(kali㉿kali)-[~]
$ ./array_ops
Enter the number of rows and columns: 2 3
Enter the matrix elements:
1 2 3
4 5 6

Menu:
1. Sum of all elements
2. Sum of each row
3. Transpose
4. Exit
Enter your choice: 1
Sum of all elements = 21
Enter your choice: 2
Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
The row-sum array contains: [6, 15]
Enter your choice: 3
Transpose of the matrix:
1      4
2      5
3      6
Enter your choice: 9
Invalid choice. Please try again.
Enter your choice: 4
Exiting...

(kali㉿kali)-[~]
$
```

Step 5: - Standard Test Table

TEST CASE	SCENARIO	INPUT	EXPECTED OUTPUT	RESULT
1	Total Sum (Option 1)	Rows: 2 Cols: 3 Elements: 1 2 3 4 5 6	<i>Sum of all elements = 21</i>	PASS
2	Row Sums (Option 2)	Same as Case 1	<i>Sum of row 1 = 6 Sum of row 2 = 15 Array contains: [6, 15]</i>	PASS
3	Transpose (Option 3)	Same as Case 1	<i>1 4 2 5 3 6 (printed with tabs)</i>	PASS
4	Out-of-bounds Choice	Choice: 9	<i>Invalid choice. Please try again.</i>	PASS
5	Exit Program	Choice: 4	<i>Exiting... (Program terminates)</i>	PASS

Step 6: - Evaluation

Reading the matrix structure *prior* to entering the *do-while* menu loop eliminates the need to request matrix data multiple times for sequential operations. Because state is preserved in local stack memory, operations like Case 1 and Case 2 calculate values seamlessly on-demand.